MyCodeSchool

exercițiu 05 — contract + upsell

Cazul tău de test din fișa scrisă de mână, automatizat cu pagini și modele · Playwright / TypeScript · Andreea

Unde lucrezi: `sap-starter/`, pe codul tău de acum (`TablePage` abstractă, cu `tableName`, `columns` și `open()`).
Ce e nou în aplicație: trei tabele, `/CFF/CONTRACTS`, `/CFF/ENT_ITEMS`, `/CFF/SCHED_ACTIONS`. Deschide-le în SE16 și uită-te la date înainte să scrii cod.
Nou, ai voie: `interface` în `models/`, o metodă care întoarce un obiect sau o listă de obiecte, `toEqual`, o funcție în `utils/`, `test.fail(...)`.
Regula rămâne: niciun `expect` în `pages/` sau `utils/`. Pagina citește, testul verifică.
Înainte să începi: `npx tsc --noEmit` fără erori.
Teorie: fișa `teorie-contract-pipeline.pdf`, secțiunile 1–5.

Pașii tăi → testele

Fișa ta	Tabel	Test
1–2, 4–5 Book contract / upsell, validate details	`/CFF/CONTRACTS`	BEGINNER
3, 6 Go to Ent: tenant, phases, request	`/CFF/ENT_ITEMS`	MIDDLE
7–8 Wait, validate „Schedule actions”	`/CFF/SCHED_ACTIONS`	EXPERIENCED

Partea 1 — modele și pagini

1. Trei modele

nivel 1

Un model spune ce formă are un rând din tabel. Fișier nou: `models/ContractDocRow.ts`.

```
export interface ContractDocRow {
  docNo: string;
  docType: string;
  productId: string;
  startDate: string;
  endDate: string;
  entId: string;
}
```

La fel, toate câmpurile `string`:

1. `models/EntItemRow.ts`: `itemNo`, `itemType`, `status`, `startDate`, `endDate`.
2. `models/ScheduledActionRow.ts`: `actionType`, `targetItem`, `status`.

Gata când: tsc nu dă nicio eroare.

2. Pagina ContractsPage

nivel 1

1. În `utils/test-data.ts`, în `tables`: `contracts`, `entItems`, `scheduledActions`, cu numele celor 3 tabele.
2. `pages/ContractsPage.ts` `extends TablePage`, cu `tableName` și `columns` (cele 7 coloane, în ordinea din SE16).
3. Trei metode:

```
async searchContract(contractId: string) {
  await this.executeTable(this.tableName, { CONTRACT_ID: contractId });
}

async readRow(rowIndex: number): Promise<ContractDocRow> {
  return {
    docNo: await this.getCellText(rowIndex, "DOC_NO"),
    docType: await this.getCellText(rowIndex, "DOC_TYPE"),
    productId: await this.getCellText(rowIndex, "PRODUCT_ID"),
    startDate: await this.getCellText(rowIndex, "START_DATE"),
    endDate: await this.getCellText(rowIndex, "END_DATE"),
    entId: await this.getCellText(rowIndex, "ENT_ID"),
  };
}

async readRows(): Promise<ContractDocRow[]> {
  const rows: ContractDocRow[] = [];
  const count = await this.getRowCount();
  for (let index = 0; index < count; index += 1) {
    rows.push(await this.readRow(index));
  }
  return rows;
}
```

4. Pagina în `fixtures/test-fixtures.ts`, ca `contractsPage`.

Gata când: tsc nu dă nicio eroare.

De reținut: numele coloanelor din SAP apar doar în pagină. Testul primește date, nu celule.

3. Celelalte două pagini

nivel 1

Același tipar ca la exercițiul 2, scris de tine:

1. `EntItemsPage`: `searchEntitlement(entId)` (filtrul `ENT_ID`), `readRow` și `readRows` cu `EntItemRow`.
2. `ScheduledActionsPage`: `searchDocument(docNo)` (filtrul `DOC_NO`), `readRow` și `readRows` cu `ScheduledActionRow`.
3. Ambele în fixtures: `entItemsPage`, `scheduledActionsPage`.

Gata când: tsc nu dă nicio eroare.

Partea 2 — cazul tău, ca test

Fișier nou: `tests/contract-upsell/contract-upsell.spec.ts`. Un tip `ContractCase` = `{ title, contractId, expected: { documents, items, actions } }`, cu tipurile din `models/`. Trei blocuri, o buclă în fiecare. Rulezi cu `npx playwright test tests/contract-upsell --project=chromium`.

4. BEGINNER — documentele contractului

nivel 1

1. Două cazuri: `C-0100` (doar booking) și `C-0200` (booking + upsell pe 30.10.2026). Valorile `expected` le iei din SE16, ca în fișa ta.
2. Test: `open()`, `searchContract(c.contractId)`, apoi întâi numărul de rânduri, apoi rândurile:

```
await expect(contractsPage.resultCount)
  .toHaveText(`${c.expected.documents.length} rows`);
expect(await contractsPage.readRows()).toEqual(c.expected.documents);
```

Indiciu: fără `resultCount` înainte, `readRows()` poate citi tabelul nefiltrat. `toEqual` compară listele câmp cu câmp.

Gata când: `2 passed`.

5. MIDDLE — Go to Ent

mai greu

1. Test cu `{ contractsPage, entItemsPage }`. Caută contractul, apoi `ENT_ID`-ul îl *citești* din primul rând: `(await contractsPage.readRow(0)).entId`. Nu-l scrii în caz.
2. `entItemsPage.open()`, `searchEntitlement(entId)`, `resultCount`, apoi `toEqual(c.expected.items)`: tenant 1000, fazele și request-urile, cu valorile din SE16.

Gata când: `4 passed`.

De reținut: ID-ul trece dintr-un tabel în altul exact ca la job. Dacă booking-ul ar crea alt entitlement, testul îl urmează singur.

6. EXPERIENCED — Schedule actions

mai greu

Pentru *ultimul* document din `c.expected.documents`: `searchDocument(docNo)`, `resultCount`, `toEqual(c.expected.actions)`.

„Wait for 2 hours” din fișa ta: aici acțiunile sunt deja rulate. Un test nu stă 2 ore pe loc; așteptarea se mută în pipeline, în exercițiul 06.

Gata când: `6 passed`.

Partea 3 — regula din fișa ta

7. Fazele se continuă fără gol

mai greu

1. Fișier nou `utils/phases.ts`, funcția `findPhaseGaps(items: EntItemRow[]): string[]`: ia doar fazele, le sortează după `startDate` și, pentru fiecare două vecine, verifică: *a doua începe a doua zi după ce se termină prima*. Dacă nu, adaugă în listă `"1001 -> 1002: 29.10.2026 -> 30.12.2026"`.
2. Test nou în EXPERIENCED, pe fiecare caz: `expect(findPhaseGaps(items)).toEqual([])` și exact o fază `Open`.
3. Adaugă al treilea caz, `C-0300`, cu aceleași valori `expected` ca `C-0200` (ENT-0300, DOC-4300 / DOC-4301). Rulează.

Indiciu: datele sunt `"zz.ll.aaaa"`; `split(".")` → `new Date(Date.UTC(an, luna - 1, zi))`, iar „a doua zi” = `setUTCDate(getUTCDate() + 1)`.

Rezultat așteptat: la `C-0300` pică 2 teste, MIDDLE și regula. Diff-ul arată `30.12.2026` în loc de `30.10.2026`.

4. E un defect în date, nu în test. Marchează-l: câmp `knownDefect?: string` în caz și, la începutul celor 2 teste, `test.fail(c.knownDefect !== undefined, c.knownDefect)`.

Gata când: `12 passed` și tsc curat.

Întrebare de răspuns în scris: cineva repară datele lui `C-0300`. Ce face testul marcat cu `test.fail` și de ce e bine așa?

De reținut: un test de date verifică și *regulile* dintre rânduri, nu doar valorile. Faza nouă începe când se termină cea veche.